AsteroidDB

分散キーバリューストアのアルゴリズム解説

目次

1. アーキテクチャ概要 — 3 プレーン構成
2. Hybrid Logical Clock (HLC) — 因果順序の追跡
3. CRDT — 衝突のない分散データ型
   - PN-Counter / LWW-Register / OR-Set / OR-Map
4. Delta Sync — 帯域効率の良いレプリケーション
5. BLS 閾値署名 — コンパクトな認証証明
6. Ack Frontier と Certification — 認証済み一貫性
7. Tag-Based Placement — レプリカ配置戦略
8. Adaptive Compaction — 自己調整型ログ圧縮

1. アーキテクチャ概要

3 プレーン・アーキテクチャ

AsteroidDB は 3 つの独立したプレーンで構成される。

プレーン	役割	主要コンポーネント
Data Plane	CRDT ベースの読み書き・レプリケーション	EventualApi, Store, DeltaSync
Authority Plane	認証フロンティア追跡・証明書発行	AckFrontier, Certificate, BLS
Control Plane	ポリシー・権限管理	SystemNamespace, PlacementPolicy

設計意図:
- Eventual 書き込みは Data Plane 内で完結（低レイテンシ）
- 認証が必要な書き込みのみ Authority Plane を経由
- ポリシー変更は Control Plane のクォーラム合意で管理

2. Hybrid Logical Clock

HLC: Hybrid Logical Clock とは

物理時計と論理時計のハイブリッド。因果順序を保証しつつ、NTP のずれにも耐える。

```
HlcTimestamp の構造:

                              (physical_ms: u64, logical: u32, node_id: NodeId)

全順序: physical_ms → logical → node_id の優先度で比較
```

なぜ物理時計だけではダメか:
- NTP の精度はミリ秒〜数十ミリ秒 → 同一ミリ秒内の複数イベントを区別できない
- クロックスキューで因果関係が逆転する可能性

HLC の解決策:
- `logical` カウンタで同一物理時刻内のイベントを順序付け
- `node_id` でタイブレーク → 完全な決定論的順序

HLC: now() と update() のアルゴリズム

now() — ローカルイベント発生時

```
fn now():
  phys = wall_clock_ms()
  if phys > self.physical:
    self.physical = phys
    self.logical  = 0
  else:
    self.logical += 1
  return (self.physical,
          self.logical,
          self.node_id)
```

update(remote) — メッセージ受信時

```
fn update(remote):
  phys = wall_clock_ms()
  if phys > max(self.phys, remote.phys):
    self.physical = phys
    self.logical  = 0
  elif self.phys == remote.phys:
    self.logical = max(self.log,
                       remote.log) + 1
  elif self.phys > remote.phys:
    self.logical += 1
  else:
    self.physical = remote.phys
    self.logical  = remote.log + 1
```

不変条件: 出力されるタイムスタンプは厳密に単調増加する（同一ノード内）

3. CRDT — Conflict-free Replicated Data Types

CRDT の基本原則

CRDT は分散環境でコンセンサスなしにデータを収束させるデータ型。

merge 演算が満たすべき 3 つの性質:

```
1. 冪等性（Idempotent）: merge(a, a) = a
2. 可換性（Commutative）: merge(a, b) = merge(b, a)
3. 結合性（Associative）: merge(merge(a, b), c) = merge(a, merge(b, c))
```

これにより:

- メッセージの重複配信を許容（冪等性）
- メッセージの到着順序不問（可換性）
- 部分的なマージでも結果は同一（結合性）

→ 最終的にすべてのレプリカが同一の状態に収束する

PN-Counter: 加減算可能な分散カウンタ

アイデア: 2 つの G-Counter（単調増加カウンタ）を組み合わせる。

データ構造:

- P: HashMap<NodeId, u64> ― ノードごとのインクリメント回数
- N: HashMap<NodeId, u64> ― ノードごとのデクリメント回数
- 値 = Σ P[i] − Σ N[i]

操作:

- increment(): P[self] += 1
- decrement(): N[self] += 1
- value(): sum(P) - sum(N)

merge アルゴリズム:

```
fn merge(other):
  for node in P ∪ other.P:
    P[node] = max(P[node],
                  other.P[node])
  for node in N ∪ other.N:
    N[node] = max(N[node],
                  other.N[node])
```

ポイント: 各ノードは自分のカウンタのみ更新 → 衝突なし / max による merge は冪等・可換・結合

PN-Counter: Delta Sync の最適化

全状態を毎回送るのは非効率 → 差分（Delta）のみを送信する。

delta_since(frontier_ts) のロジック:

```
fn delta_since(frontier):
  delta = PnCounter::new()
  for (node, count) in self.P:
    if self.last_updated[node] > frontier:
      delta.P[node] = count
  for (node, count) in self.N:
    if self.last_updated[node] > frontier:
      delta.N[node] = count
  return delta
```

- Frontier（最後に同期した HLC タイムスタンプ）以降の変更のみ抽出
- 受信側は通常の `merge` で `delta` を適用 → 同じ `merge` 関数を再利用

LWW-Register: 最終書き込み勝ちレジスタ

単一の値を保持し、タイムスタンプが最新の書き込みが勝つ。

データ構造: `value: Option<T>` — 現在の値 / `timestamp: HlcTimestamp` — 最終書き込み時刻

書き込み:

```
fn set(val, ts):
  if ts > self.timestamp:
    self.value = Some(val)
    self.timestamp = ts
```

merge:

```
fn merge(other):
  if other.timestamp > self.timestamp:
    self.value = other.value
    self.timestamp = other.timestamp
  // タイブレーク:
  // physical → logical → node_id
```

タイブレークの重要性:

- HLC の全順序保証により、同一タイムスタンプは存在しない
- `node_id` が最終的なタイブレーカ → 決定論的で可換

OR-Set: Observed-Remove Set (1/2)

Add-wins セマンティクス: 同時の追加と削除が衝突した場合、追加が勝つ。

データ構造:

- `entries: HashMap<Element, HashSet<Dot>>` — 要素ごとのドット集合
- `clock: HashMap<NodeId, u64>` — ノードごとの単調カウンタ
- `deferred: HashSet<Dot>` — 因果コンテキスト（トゥームストーン）
- `Dot = (node_id, counter)` — 各追加操作のユニーク識別子

操作:

add(elem): 新しいドットを生成

```
counter[self] += 1
entries[elem].insert(
  (self, counter[self]))
```

remove(elem): ドットを deferred に移動

```
deferred ∪= entries[elem]
entries.remove(elem)
```

OR-Set: Observed-Remove Set (2/2)

merge アルゴリズム:

```
fn merge(other):
  for elem in self.entries ∪ other.entries:
    self_dots  = self.entries[elem]
    other_dots = other.entries[elem]
    // 相手側で削除されていないドットを残す
    kept = (self_dots ∩ other_dots)
         ∪ (self_dots \ other.deferred)
         ∪ (other_dots \ self.deferred)
    if kept.is_empty():
      self.entries.remove(elem)
    else:
      self.entries[elem] = kept
  self.deferred ∪= other.deferred
```

なぜ Add-wins になるか:

- `remove` は「観測済みのドット」のみを削除
- 同時に発生した `add` は新しいドット → `deferred` に含まれない → 残る
- GC: `compact_deferred()` で古いトゥームストーンを回収可能

OR-Map: OR-Set + LWW-Register のハイブリッド

キーの存在は OR-Set セマンティクス、値は LWW-Register で管理。

データ構造:

- entries: HashMap<Key, (HashSet<Dot>, LwwRegister<Value>)>
- clock: HashMap<NodeId, u64> / deferred: HashSet<Dot>

merge のポイント:

```
fn merge(other):
  for key in all_keys:
    // 1. ドット集合のマージ (OR-Set と同様)
    kept_dots = or_set_merge(self.dots[key], other.dots[key])
    // 2. 値のマージ (LWW-Register)
    merged_value = lww_merge(self.value[key], other.value[key])
    // 3. キーが存在する場合のみ保持
    if kept_dots.is_empty():  entries.remove(key)
    else:  entries[key] = (kept_dots, merged_value)
```

→ 同時の set と delete → Add-wins（キーが残り、最新の値が使われる）

4. Delta Sync プロトコル

Delta Sync: フロンティアベースの差分同期

各ピアとの最終同期点（Frontier）を記憶し、差分のみを転送する。

```
プロトコルフロー:
1. Requester → Responder:
   DeltaSyncRequest { frontier: HlcTimestamp }

2. Responder:
   changed = entries.filter(|e| e.timestamp > frontier)
   if changed.len() / total.len() > THRESHOLD:
     return FullSync(all_entries)    // フルシンクにフォールバック
   else:
     return DeltaSync(changed)       // 差分のみ

3. Requester:
   for entry in response:
     store.merge(entry)              // CRDT merge を適用
   update_frontier(responder, now())
```

フォールバック閾値: 変更率 > 50% なら全体送信の方が効率的（デフォルト）

Delta Sync: Exponential Backoff with Jitter

ネットワーク障害時の再試行戦略。Thundering Herd（雷群問題）を防止する。

```
initial_delay = 500ms
max_delay     = 2000ms
jitter_factor = 0.25

fn next_backoff(current_delay):
  base = min(current_delay * 2, max_delay)
  jitter = random(0, base * jitter_factor)
  return base + jitter

// 成功時: delay をリセット
// 失敗時: delay = next_backoff(delay)
```

各ピアごとに独立したバックオフ状態を保持:

- ピア A が応答不能でも、ピア B との同期は影響を受けない
- ネットワークエラーとデシリアライゼーションエラーを区別

5. BLS 閾值署名

BLS12-381 Aggregate Signatures

N 人の署名者による署名を 1 つの署名に集約できる。

数学的基盤: 双線形ペアリング $e : G_1 \times G_2 \to G_T$

- 鍵生成: 秘密鍵 $s_k \in \mathbb{Z}_p$, 公開鍵 $p_k = s_k \cdot g_2$
- 署名: $\sigma_i = s_{k_i} \cdot H(m)$ $(H : \{0,1\}^* \to G_1)$
- 集約: $\sigma_{\text{agg}} = \sum \sigma_i$
- 検証: $e(\sigma_{\text{agg}}, g_2) = e\left(H(m), \sum p_{k_i}\right)$

AsteroidDB での活用:

従来 (Ed25519 個別署名):

- N 個の署名 × 64 bytes
- 証明サイズ: O(N)

BLS 集約署名:

- 1 つの集約署名 = 48 bytes
- 証明サイズ: O(1) (定数)

→ Authority 数が増えてもネットワーク帯域・ストレージコストが増加しない

証明書管理: デュアルモード

Ed25519 と BLS の共存によるローリングアップグレードをサポート。

```
Certificate 構造:
enum CertificateSignature {
  Ed25519 { signer, signature },        // 個別署名
  BlsAggregate { signers, signature },   // 集約署名
}
```

エポックベースの鍵ローテーション:

- KeysetRegistry がエポックごとの鍵セットを管理
- 猶予期間（Grace Period）中は旧・新どちらの鍵でも検証可能
- フォーマットバージョンで互換性を維持

6. Ack Frontier と Certification

Certified 一貫性の仕組み

Eventual と Certified の 2 つの一貫性レベルを統一する。

Eventual 書き込み: ローカルに即時受理 → CRDT で非同期収束

Certified 書き込み: ローカルに受理 → 過半数の Authority が確認したら認証済み

Ack Frontier の概念:

```
Authority A が frontier = T と報告
  → 「A はタイムスタンプ T 以前のすべての書き込みを処理済み」

過半数 Frontier = max(T) s.t. |{A : frontier(A) >= T}| > N/2
  → 「過半数の Authority が T 以前を確認済み」

書き込み W の認証条件:
  W.timestamp <= 過半数 Frontier → Certified ✓
```

Certification フロー

```
[Client]                    [Node]                  [Authorities]
   |                          |                          |
   |--- POST /certified ---->|                          |
   |                          |-- store locally -------->|
   |                          |   status: Pending        |
   |                          |                          |
   |                          |<-- ack frontier reports -|
   |                          |   A1: T=100, A2: T=95   |
   |                          |   A3: T=102             |
   |                          |                          |
   |                          |-- check majority ------->|
   |                          |   write_ts=98            |
   |                          |   majority_frontier=100  |
   |                          |   98 <= 100 → Certified! |
   |                          |                          |
   |<-- GET /certified ------|                          |
   |    value + ProofBundle   |                          |
   |    + BLS Certificate     |                          |
```

Fencing: ポリシーバージョンが変更された場合、古いフロンティアは無効化される

7. Tag-Based Placement

レプリカ配置のアルゴリズム

ノードのタグに基づいてレプリカの配置先を決定する。

```
PlacementPolicy:
struct PlacementPolicy {
  replica_count: usize,
  required_tags: HashSet<Tag>,    // 必須タグ（AND 条件）
  forbidden_tags: HashSet<Tag>,   // 禁止タグ
  max_latency_ms: Option<u64>,    // レイテンシ制約
}
```

ノード選択アルゴリズム:

```
fn select_nodes(policy, topology):
  candidates = topology.nodes.filter(|node|
    policy.required_tags  ⊆ node.tags        // 必須タグをすべて持つ
    ∧ policy.forbidden_tags ∩ node.tags = ∅   // 禁止タグを持たない
    ∧ (policy.max_latency_ms.is_none()
       ∨ node.latency <= policy.max_latency_ms))
  candidates.sort_by(|a, b| a.node_id.cmp(b.node_id))  // 決定論的
  return candidates[..policy.replica_count]
```

ユースケース: `required: [region:asia, tier:ssd]` → アジアリージョンの SSD ノードのみ

Rebalance: ポリシー変更時のレプリカ移動

ポリシー変更やノード追加/削除時にレプリカを再配置する。

```
Rebalance プランニング:
fn plan_rebalance(old_nodes, new_nodes):
  to_add    = new_nodes \ old_nodes  // 新たに追加すべきレプリカ
  to_remove = old_nodes \ new_nodes  // 削除すべきレプリカ

  plan = []
  for node in to_add:
    plan.push(Move::Add(node))
  for node in to_remove:
    plan.push(Move::Remove(node))
  return plan
```

- 新ノードへのデータ転送完了後に旧ノードからデータを削除
- ポリシーバージョンによるフェンシングで安全性を保証

8. Adaptive Compaction

ログ圧縮とチェックポイント

CRDT のマージログは際限なく増加する → 定期的な圧縮が必要。

```
チェックポイントのトリガー条件:
fn should_compact(state):
  state.ops_since_checkpoint > ops_threshold
  v state.elapsed_since_checkpoint > time_threshold
```

Adaptive Tuner — 自己調整メカニズム:

```
fn adjust_thresholds(frontier_lag):
  if frontier_lag > high_watermark:
    // Authority の処理が遅れている → 圧縮頻度を上げる
    ops_threshold  *= 0.8
    time_threshold *= 0.8
  elif frontier_lag < low_watermark:
    // 余裕がある → 圧縮頻度を下げる（リソース節約）
    ops_threshold  *= 1.2
    time_threshold *= 1.2
```

フロンティアラグ: Authority がまだ確認していない最古の書き込みからの経過時間

9. まとめ

AsteroidDB のアルゴリズム全体像

コンポーネント	アルゴリズム・技術
時刻管理	Hybrid Logical Clock — 因果順序 + NTP 耐性
データ収束	CRDT (PN-Counter, LWW-Register, OR-Set, OR-Map)
レプリケーション	Delta Sync — フロンティアベース差分転送
認証	BLS12-381 集約署名 — O(1) 証明サイズ
一貫性	Ack Frontier — 過半数フロンティアによる認証
配置	Tag-Based Placement — 決定論的レプリカ選択
運用	Adaptive Compaction — フロンティアラグ追従型自己調整

核心的な設計思想:

- Eventual + Certified の二重一貫性 → ユースケースに応じた柔軟な選択
- CRDT による Local-first → 書き込みは常にローカルで即時受理
- BLS 集約署名 → スケーラブルな認証証明
- 高レイテンシ耐性 → 衛星通信のような環境にも対応